

Skill-16

Rollno:2520030011

Program:

```
tejaswini@teju: ~/OS_Lab/skil × + v
GNU nano 8.7.1
// OSSP Skill 16 - Background Jobs
// Demonstrates &, non-blocking child processes, job storage,
// monitoring with waitpid(WNOHANG), and job cleanup.

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 16

typedef struct {
    int id;
    pid_t pid;
    int active;
} Job;

static Job jobs[MAX_JOBS];
static int next_id = 1;

static void add_job(pid_t pid) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active) {
            jobs[i].id = next_id++;
            jobs[i].pid = pid;
            jobs[i].active = 1;
            printf("[%d] %d started in background\n", jobs[i].id, pid);
            return;
        }
    }
    printf("Job table is full.\n");
}
```

```

GNU nano 8.7.1
static void update_jobs(void) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active)
            continue;

        int status;
        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

        if (result == jobs[i].pid) {
            printf("[%d] pid %d completed\n", jobs[i].id, jobs[i].pid);
            jobs[i].active = 0;
        }
    }
}

static void list_jobs(void) {
    update_jobs();

    for (int i = 0; i < MAX_JOBS; i++)
        if (jobs[i].active)
            printf("[%d] Running pid=%d\n", jobs[i].id, jobs[i].pid);
}

int main(void) {
    char line[128];

    printf("=== Skill 16: Background Jobs ===\n");
    printf("Commands: sleep N & | jobs | exit\n");

    while (1) {
        update_jobs();

        printf("bg> ");
        if (!fgets(line, sizeof(line), stdin))
            break;

        line[strcspn(line, "\n")] = '\0';

        if (strcmp(line, "exit") == 0)
            break;

        if (strcmp(line, "jobs") == 0) {
            list_jobs();
            continue;
        }

        int seconds;
        if (sscanf(line, "sleep %d &", &seconds) == 1) {
            pid_t pid = fork();

            if (pid == -1) {
                perror("fork");
            } else if (pid == 0) {
                sleep(seconds);
                _exit(0);
            } else {
                add_job(pid);
            }
        } else {
            printf("Use: sleep N &\n");
        }
    }
}

```

```

    // Reap remaining children before exit.
    for (int i = 0; i < MAX_JOBS; i++)
        if (jobs[i].active)
            waitpid(jobs[i].pid, NULL, 0);

    return 0;
}

```

Output:

```

tejaswini@teju:~/OS_Lab/skill$ nano skill_16.c.c
tejaswini@teju:~/OS_Lab/skill$ gcc skill_16.c.c -o skill_16
tejaswini@teju:~/OS_Lab/skill$ ./skill_16
=== Skill 16: Background Jobs ===
Commands: sleep N & | jobs | exit
bg> sleep 5 &
[1] 1315 started in background
bg> jobs
[1] pid 1315 completed
bg> jobs
bg> sleep 3 &
[2] 1316 started in background
bg> jobs
[2] pid 1316 completed
bg> exit
tejaswini@teju:~/OS_Lab/skill$ |

```